Bài 1. Minimax và Alpha-Beta Pruning

Cây trò chơi gồm nút gốc thuộc MAX, ba nút MIN ở tầng kế tiếp, và mỗi nút MIN có ba nút MAX con. Mỗi nút MAX ở tầng lá có hai lá, tổng cộng 18 lá.

a) Giá trị Minimax của các nút trung gian.

- Nhánh trái: M1 = max(3, 0) = 3, M2 = max(−1, 1) = 1, M3 = max(4, 6) = 6. Do đó N1 = min(3, 1, 6) = 1.
- Nhánh giữa: M4 = max(10, −1) = 10, M5 = max(0, −2) = 0, M6 = max(12, −3) = 12. Do đó N2 = min(10, 0, 12) = 0.
- Nhánh phải: M7 = max(−4, −2) = −2, M8 = max(−2, 10) = 10, M9 = max(−5, −7) = −5. Do đó N3 = min(−2, 10, −5) = −5.

Nút gốc MAX nhận giá trị max(1, 0, −5) = 1.

b) Nhánh được chọn và giá trị trò chơi. Nút gốc nên chọn nhánh trái (nút N1), vì N1 có giá trị lớn nhất trong ba nhánh. Giá trị của trò chơi là **1**.

c) Alpha-Beta Pruning, duyệt từ trái sang phải. Ký hiệu (α, β) là cửa sổ tại mỗi nút khi được thăm.

- Nút gốc: (α, β) = (−∞, +∞). Chuyển xuống N1.
- N1 (MIN): (−∞, +∞). Duyệt M1 với cửa sổ (−∞, +∞), thu được 3, nên β của N1 = 3. Duyệt M2 với cửa sổ (−∞, 3), thu được 1, nên β của N1 = 1. Duyệt M3 với cửa sổ (−∞, 1): lá 4 có giá trị 4 ≥ β = 1, nên lá 6 bị cắt tỉa (đánh dấu X). Giá trị N1 = 1, và α của nút gốc trở thành 1.
- N2 (MIN): (1, +∞). Duyệt M4 với cửa sổ (1, +∞), thu được 10, nên β của N2 = 10. Duyệt M5 với cửa sổ (1, 10), thu được 0. Vì giá trị của N2 là 0 ≤ α = 1, phần còn lại của N2 bị cắt tỉa, tức là M6 (đánh dấu X). Giá trị N2 = 0.
- N3 (MIN): (1, +∞). Duyệt M7 với cửa sổ (1, +∞), thu được −2. Vì β của N3 = −2 ≤ α = 1, hai nút M8 và M9 bị cắt tỉa (đánh dấu X). Giá trị cận trả về là −2 không ảnh hưởng đến kết quả ở nút gốc.

Giá trị nút gốc là 1, trùng với kết quả Minimax.

d) Các lá không cần thăm. Có 7 lá không cần thăm: lá 6; lá 12 và −3; lá −2 và 10; lá −5 và −7. Minimax đánh giá toàn bộ 18 lá, trong khi Alpha-Beta chỉ đánh giá 11 lá và vẫn cho cùng giá trị gốc. Số lá được cắt tỉa phụ thuộc vào thứ tự duyệt. Nếu các nút tốt được duyệt trước, số lá cắt tỉa có thể tăng thêm.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expectiminimax)

Trong ba cây của Hình 2, nút gốc A thuộc MIN, nút B thuộc MAX và nút C là nút ngẫu nhiên có hai nhánh với xác suất bằng nhau. Do đó giá trị của nút C bằng trung bình cộng của hai con, tức là C = (x + y)/2. Các nhánh được duyệt từ trái sang phải. Quy ước của đề bài là không thực hiện cắt tỉa khi hai giá trị bằng nhau.

a) Trường hợp 1 (cây A → B → C). Lá viền đậm là lá thứ bảy, tức lá trái của nút C thứ tư. Một cách điền minh họa là: C1 có hai lá (2, 2), C2 có hai lá (4, 4), C3 có hai lá (6, 6).

- Nhánh B trái: B1 = max(C1, C2) = max(2, 4) = 4. Sau bước này, β của gốc A = 4.
- Nhánh B phải: C3 = (6 + 6)/2 = 6. Vì B2 ≥ C3 = 6 > β = 4, nút A (MIN) không chọn nhánh này. Do đó nút C4, cùng lá viền đậm, bị cắt tỉa.

Điều kiện cắt tỉa: giá trị trung bình của C3 lớn hơn giá trị B1 đã biết, tức là (lá 5 + lá 6)/2 > B1. Giá trị gốc là 4.

b) Trường hợp 2 (cây A → C → B). Lá viền đậm là lá thứ tám, tức lá phải nhất, thuộc nút B thứ tư. Cây có C1 gồm B1 và B2, C2 gồm B3 và B4.

- Điền minh họa: B1 = max(2, 2) = 2, B2 = max(2, 2) = 2, nên C1 = (2 + 2)/2 = 2. Với nhánh C2, B3 = max(5, 3) = 5 và lá đầu tiên của B4 là 0.
- Khi đã có B3 = 5 và lá đầu của B4 là 0, ta có C2 = (B3 + B4)/2 ≥ (5 + 0)/2 = 2.5 > β = 2 của gốc A. Vì vậy lá viền đậm bị cắt tỉa.

Điều kiện cắt tỉa: (B3 + lá 7)/2 > C1. Giá trị gốc là 2.

c) Trường hợp 3 (cây A → C → B). Lá viền đậm là lá thứ hai của B3. **Không thể cắt tỉa** với các giá trị hữu hạn đã biết.

Để cắt lá này, cần C2 ≥ C1, tức (B3 + B4)/2 ≥ C1. Tại thời điểm duyệt lá viền đậm, chỉ có cận dưới B3 ≥ lá 5, còn B4 chưa được thăm và chưa có cận nào. Nếu B4 có thể nhận

giá trị nhỏ tùy ý, thì C2 có thể nhỏ hơn C1 và nhánh không thể bị cắt. Do đó, cắt tỉa chỉ khả thi khi biết cận dưới chung L cho mọi lá, và điều kiện là (lá 5 + L)/2 ≥ C1.

d) Vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax. Trong nút MIN hoặc MAX, giá trị là min hoặc max của các con, nên chỉ cần một con đã thăm vượt ngưỡng là đủ để cắt tỉa. Trong nút ngẫu nhiên, giá trị là trung bình có trọng số. Một nhánh chưa thăm có thể kéo trung bình lên hoặc xuống bất kỳ mức nào. Vì vậy, chỉ có thể cắt khi đã biết cận từ các nhánh đã thăm, hoặc khi có cận chung cho các nhánh chưa thăm. Ngoài ra, cận chỉ truyền được qua nút MIN và MAX theo chiều đơn điệu, nên cần suy luận thêm để áp dụng cho nút ngẫu nhiên.

Bài 3. Câu hỏi đúng/sai

Gọi V_M là giá trị Minimax của cây gốc MAX, và V_E là giá trị Expectimax của cây sau khi thay mọi nút MIN bằng nút ngẫu nhiên đã biết phân phối xác suất.

a) Đúng. Với nút ngẫu nhiên có xác suất $p_i \geq 0$ và $\Sigma p_i = 1$, ta có $\Sigma p_i \cdot v_i \geq \min_i v_i$. Mỗi nút MIN được thay bằng nút ngẫu nhiên có giá trị không nhỏ hơn min các con. Nút MAX là hàm đơn điệu theo các con. Bằng quy nạp từ lá lên gốc, V_E ≥ V_M.

b) Đúng. Chính sách Minimax tối ưu đảm bảo giá trị ở mỗi nhánh con không nhỏ hơn V_M, vì V_M là giá trị an toàn khi đối thủ chọn phương án xấu nhất. Tại mỗi nút ngẫu nhiên, giá trị kỳ vọng là trung bình của các nhánh, và mỗi nhánh đều ≥ V_M, nên trung bình cũng ≥ V_M. Bằng quy nạp, lợi ích kỳ vọng ít nhất là V_M.

c) Sai. Xét phản ví dụ sau. Gốc MAX có hai nút MIN là X và Y. Nút X có hai lá 1 và 10, nút Y có hai lá 0 và 3. Minimax cho V_M = max(min(1, 10), min(0, 3)) = 1, và chính sách Minimax chọn X. Khi thay X bằng nút ngẫu nhiên với xác suất (0.9, 0.1) cho các lá (1, 10), kỳ vọng là 1.9. Khi thay Y bằng nút ngẫu nhiên với xác suất (0.1, 0.9) cho các lá (0, 3), kỳ vọng là 2.7. Khi đó V_E = max(1.9, 2.7) = 2.7. Chính sách Minimax đạt kỳ vọng 1.9 < 2.7 = V_E, nên mệnh đề sai.

d) Lựa chọn phương pháp. Trong ba phương án, các đánh dấu được xác định như sau:

- Minimax với tiện ích +1/−1: **không phù hợp**. Minimax giả định một đối thủ luôn chọn phương án gây bất lợi nhất, trong khi bài toán chỉ có một người chơi và kết quả là ngẫu nhiên.

- Expectimax với tiện ích +1/−1: **phù hợp**. Giả định cần có là mô hình xác suất các kết quả đã biết, không gian trạng thái hữu hạn, và người chơi tối đa hóa kỳ vọng.
- Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả: **phù hợp**. Giả định cần có là mô hình MDP với hàm chuyển trạng thái đã biết, tính Markov, quan sát đầy đủ, và chiết khấu hoặc chân trời hữu hạn để đảm bảo hội tụ.
- Không phương án nào: không đúng, vì hai phương án trên đã thỏa mãn.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

a) Đồ thị ràng buộc. Các đỉnh là sáu biến A, B, C, D, E, F. Các cạnh là các ràng buộc nhị phân: A–B, A–C, A–D, B–C, B–E, C–F. Láng giềng của từng biến như sau:

- A: B, C, D (bậc 3).
- B: A, C, E (bậc 3).
- C: A, B, F (bậc 3).
- D: A (bậc 1); E: B (bậc 1); F: C (bậc 1).

Đồ thị có một chu trình tam giác A–B–C, nên không phải là cây.

b) Forward Checking sau khi gán A. Các biến chưa gán là láng giềng của A, nên miền của **B, C, D** có thể bị thay đổi.

c) Forward Checking tại B, sau khi đã gán A. Láng giềng chưa gán của B là C và E, còn A đã được gán. Vì vậy miền của **C và E** có thể bị thay đổi.

d) Duy trì Arc Consistency sau khi gán A. Các biến B, C, D bị lọc trực tiếp theo A. Việc lọc B và C kéo theo lan truyền: E có thể bị lọc qua B, và F có thể bị lọc qua C. Vậy miền của **B, C, D, E, F** có thể thay đổi. Biến A không thay đổi vì đã được gán.

e) Duy trì Arc Consistency sau khi gán A, rồi gán B. Sau khi gán B, miền của C và E bị lọc theo B, và lọc C tiếp tục lan sang F. Biến D chỉ chịu ràng buộc với A, nên không thay đổi thêm. Vậy miền của **C, E, F** có thể thay đổi.

f), g), h) Số lần quay lui trong trường hợp xấu nhất. Đồ thị ràng buộc của Hình 3 là một cây gồm sáu cạnh: A–B, A–C, B–D, D–F, C–E, E–G. Thuật toán chạy AC ban đầu,

và chạy lại sau khi đã gán 2, 4 và 6 biến. Vì vậy bảy biến được chia thành bốn khối: {biến 1, 2}, {biến 3, 4}, {biến 5, 6} và {biến 7}. Các giả định được sử dụng như sau:

- Mỗi biến có miền kích thước d, và mỗi ràng buộc là nhị phân.
- Mỗi lần thử một giá trị bị loại vì xung đột với biến đã gán, hoặc khiến một miền bị rỗng khi AC chạy, được tính là một lần quay lui.
- Trường hợp xấu nhất: đối thủ chọn ràng buộc sao cho số lần thử sai là tối đa, nhưng bài toán vẫn có nghiệm.

Lập luận gồm hai loại nguồn quay lui:

- **Xung đột trong cùng khối.** Nếu hai biến có cạnh nối và cùng thuộc một khối, biến được gán sau không được lọc theo biến trước cho đến lần AC tiếp theo. Với mỗi giá trị của biến trước, AC ở đầu khối bảo đảm có ít nhất một giá trị hỗ trợ của biến sau. Do đó tối đa $d - 1$ giá trị của biến sau bị loại. Mỗi cạnh trong khối gây tối đa $d(d - 1)$ lần quay lui, và cận này đạt được.
- **Biến có hai láng giềng đã gán.** Nếu một biến chưa gán có hai láng giềng đã được gán, AC sau mốc gần nhất có thể làm miền của nó rỗng. Khi đó mỗi tổ hợp sai của hai láng giềng gây một lần quay lui. Số tổ hợp sai tối đa là $d^2 - 1$, vì ít nhất một tổ hợp phải đúng.

Áp dụng cho từng thứ tự:

- **Thứ tự f** (A, B, C, D, E, F, G): khối {A, B} chứa cạnh A–B, nên có một cạnh trong khối. Không biến nào có hai láng giềng đã gán. Tổng cộng: $d(d - 1) = d^2 - d$.
- **Thứ tự g** (G, E, C, A, B, D, F): các khối {G, E}, {C, A} và {B, D} mỗi khối chứa một cạnh, nên có ba cạnh trong khối. Không biến nào có hai láng giềng đã gán. Tổng cộng: $3(d^2 - d)$.
- **Thứ tự h** (G, F, E, D, C, B, A): không khối nào chứa cạnh, nên không có xung đột trong khối. Biến A có hai láng giềng đã gán là B và C, nên AC sau mốc 6 có thể làm rỗng miền của A. Tổng cộng: $d^2 - 1$.

Kết luận: Ba thứ tự cho ba cận khác nhau: f cho $d^2 - d$, h cho $d^2 - 1$, và g cho $3(d^2 - d)$, là cận lớn nhất. Nhận định trước đây cho rằng ba thứ tự có cùng cận là sai, vì thứ tự g có ba cạnh trong khối và thứ tự h có một biến với hai láng giềng đã gán. Nếu quy ước

đếm quay lui khác đi, các hằng số có thể thay đổi, nhưng thứ tự g vẫn cho số lần quay lui lớn nhất.

Bài 5. Cây trò chơi Pacman và Casper

Trong cây, mỗi nút có cặp (u, v). Ở đây u là giá trị mà Pacman tin tưởng, còn v là giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối. Nút Casper chọn min theo u và max theo v. Nút Pacman chọn con có u lớn nhất và lấy cặp của con đó.

a) Điền các cặp còn thiếu.

- Nút X thuộc Pacman, có hai con (4, 6) và (1, 7). Chọn (4, 6) vì 4 > 1.
- Nút Y thuộc Pacman, có hai con (0, 6) và (3, 5). Chọn (3, 5) vì 3 > 0.
- Nút Casper bên trái thứ nhất, có lá (2, 2) và X = (4, 6). Cặp là (min(2, 4), max(2, 6)) = (2, 6).
- Nút Casper bên trái thứ hai, có lá (1, 1) và Y = (3, 5). Cặp là (min(1, 3), max(1, 5)) = (1, 5).
- Nút Pacman bên trái, có hai con (2, 6) và (1, 5). Chọn (2, 6) vì 2 > 1.
- Nút Pacman bên phải có hai con là các nút Casper (3, 9) và (4, 8). Chọn con có u lớn nhất là 4, tức (4, 8).
- Nút gốc, có hai con (2, 6) và (4, 8). Cặp là (min(2, 4), max(6, 8)) = (2, 8).

Khác biệt giữa u và v tại nút Casper. u là niềm tin của Pacman. Pacman tin rằng Casper luôn làm giảm tiện ích, nên lấy min theo u. Trong thực tế, Casper thân thiện và muốn tăng tiện ích, nên v lấy max. Do đó v thường lớn hơn hoặc bằng u.

b) Các nhánh có thể không cần duyệt. Duyệt từ trái sang phải:

- Trong nút (1, 7), sau khi thăm lá (1, 1) có u = 1 < 4, là u của X mà Pacman đã chọn. Vì vậy lá (7, 7) không cần thăm.
- Trong nhánh Y, sau khi thăm lá (1, 1) có u = 1 < 2, là α đã có ở nút Pacman bên trái. Vì vậy nhánh Y, gồm các lá (6, 6), (0, 0), (3, 3), (5, 5), không cần thăm.

Tổng cộng, lá (7, 7) và toàn bộ nhánh Y có thể không cần duyệt.

c) Trường hợp Pacman biết Casper thân thiện. Khi đó Pacman tin rằng Casper cũng tối đa hóa tiện ích, nên mọi nút đều dùng max theo cùng một giá trị. Giá trị gốc là max

của mọi lá, tức là **9**. Giá trị gốc tăng từ 8 lên 9, và đường đi tối ưu là gốc → nhánh phải → (3, 9) → lá (9, 9).

Bài 6. Thuật toán tìm kiếm: Tính đầy đủ và tối ưu

Bảng sau tổng hợp điều kiện để mỗi thuật toán đảm bảo tính đầy đủ và tính tối ưu. Trên đồ thị, cần kiểm tra trạng thái đã thăm để tránh lặp.

Thuật toán	Điều kiện đầy đủ (Complete)	Điều kiện tối ưu (Optimal)
Breadth-First Search (BFS)	Có, nếu hệ số nhánh b hữu hạn (trên cây), hoặc không gian trạng thái hữu hạn có kiểm tra trạng thái đã thăm (trên đồ thị).	Có, khi mọi bước có chi phí bằng nhau (hoặc chi phí không giảm theo độ sâu).
Depth-First Search (DFS)	Không tổng quát: có thể đi vào nhánh vô hạn. Đầy đủ trên cây hữu hạn, hoặc trên đồ thị hữu hạn có kiểm tra trạng thái lặp.	Không, trừ trường hợp đặc biệt.
Iterative Deepening Search (IDS)	Có, nếu b hữu hạn (tương tự BFS).	Có, khi mọi bước có chi phí bằng nhau.
A* Search	Có, nếu b hữu hạn và chi phí mỗi bước $\geq \varepsilon > 0$ (trên cây); có, nếu không gian trạng thái hữu hạn (trên đồ thị).	Trên cây: heuristic chấp nhận được (admissible). Trên đồ thị: heuristic nhất quán (consistent).

BFS, DFS và IDS không sử dụng chi phí bước khi chọn nút mở rộng, nên chỉ tối ưu khi chi phí các bước bằng nhau. Với A*, nếu chi phí bước có thể tiến về 0 thì thuật toán có thể không tiến đến đích, nên cần $\varepsilon > 0$ để đảm bảo tính đầy đủ. Heuristic chấp nhận được không vượt quá chi phí thật, nên A* tối ưu trên cây. Trên đồ thị, cần heuristic nhất quán để một nút đã đóng không phải mở lại. Nếu heuristic vượt quá chi phí thật, A* có thể trả về một đường không tối ưu.

Bài 7. A* khi chi phí một cạnh thay đổi

Gọi C* là chi phí đường tối ưu hiện tại. Giả sử heuristic vẫn hợp lệ sau khi thay đổi chi phí cạnh X → Y.

a) Chi phí tăng n; cạnh thuộc đường tối ưu cũ và đã được khám phá. Đường cũ có chi phí tăng, nên có thể có đường khác tốt hơn. Không cần chạy lại từ đầu. Cần cập nhật g(Y) và các hậu duệ phụ thuộc vào Y, sau đó mở lại các nút này và tiếp tục A*.

b) Chi phí giảm n; cạnh thuộc đường tối ưu cũ và đã được khám phá. Đường cũ vẫn là tối ưu với chi phí $C^* - n$. Mọi đường khác không đi qua cạnh này có chi phí không nhỏ hơn C^*, nên chỉ cần cập nhật chi phí đường. Cần lưu ý rằng giảm chi phí có thể làm heuristic không còn hợp lệ, nên phải kiểm tra lại.

c) Chi phí tăng n; cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá. Đường tối ưu không đổi. Chỉ cần kiểm tra xem $g(Y)$ có đang lấy giá trị từ cạnh $X \rightarrow Y$ hay không. Nếu có, tính lại $g(Y)$ từ các nút cha khác và lan truyền.

d) Chi phí giảm n; cạnh không thuộc đường tối ưu cũ nhưng đã được khám phá. Đường qua cạnh này có thể rẻ hơn. Kiểm tra $g(X) + (c - n) < g(Y)$. Nếu đúng, cập nhật $g(Y)$, mở lại Y, lan truyền xuống các hậu duệ và tiếp tục A* từ hàng đợi hiện tại. Đường mới tốt hơn nếu chi phí đến đích nhỏ hơn C^*.

e) Chi phí tăng n; cạnh không thuộc đường tối ưu cũ và chưa được khám phá. Không ảnh hưởng đến kết quả. Cạnh này chưa được sử dụng, và tăng chi phí chỉ làm nó kém hấp dẫn hơn.

f) Chi phí giảm n; cạnh không thuộc đường tối ưu cũ và chưa được khám phá. Có thể tạo ra đường tốt hơn. Nếu X chưa được mở rộng, cạnh mới sẽ được xét khi X được mở. Nếu A* đã dừng, cần kiểm tra $g(X) + (c - n) + h(Y) < C^*$. Nếu đúng, tiếp tục tìm kiếm từ X.

Bài 8. Tác tử thông minh và môi trường

Tác tử A chọn hành động không phụ thuộc vào percept. Tác tử B chỉ phụ thuộc vào percept hiện tại. Tác tử C sử dụng toàn bộ lịch sử percept. Để hành động hợp lý hoàn hảo trong mọi môi trường của một lớp, hành động tối ưu phải được quyết định bởi thông tin mà tác tử có thể sử dụng.

a) Quan sát đầy đủ, xác định, rời rạc, một tác tử, tĩnh. Hành động tối ưu phụ thuộc vào trạng thái hiện tại, và các môi trường trong lớp có trạng thái tối ưu khác nhau. Tác tử A không đủ. Tác tử B đủ, vì percept đã chứa trạng thái đầy đủ. Tác tử C cũng đủ, vì có thể bỏ qua phần lịch sử không cần thiết. **Các lớp: B và C.**

b) Quan sát một phần, xác định, rời rạc, một tác tử, tĩnh. Cùng một percept có thể đòi hỏi hành động khác nhau tùy vào lịch sử, vì trạng thái ẩn có thể khác nhau. Tác tử B không đủ. Chỉ tác tử C sử dụng được lịch sử. **Lớp: C.**

c) Quan sát một phần, ngẫu nhiên, rời rạc, một tác tử, tĩnh. Vẫn cần lịch sử để suy ra niềm tin về trạng thái hiện tại. Chỉ tác tử C đủ. **Lớp: C.**

d) Phát biểu "có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý": Đúng. Ví dụ, xét một môi trường có tiện ích hằng số, tức là mọi hành động đều cho cùng một kết quả. Khi đó mọi hành động đều tối ưu, nên mọi tác tử, kể cả A, đều hành động hợp lý.